class07: Machine Learning 1

Niam Shasha A17805610

Table of contents

Background

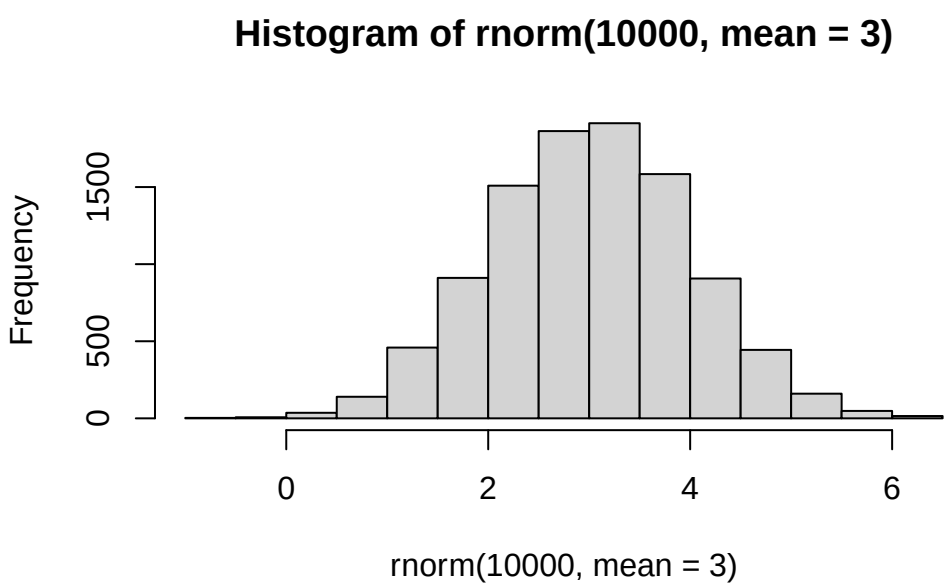
Today we will explore core machine learning methods thst are very popular in bioinformatics. These include **clustering** and **deminsionallity** reduction.

Kmeans clustering

The main function inn "base" R for k-means clustering called `kmeans()`

Before we go too deep let's make up some "simple" data that we can cluster and know if we are getting a good answer or not. To help us do this w can use the `rnorm()` function.

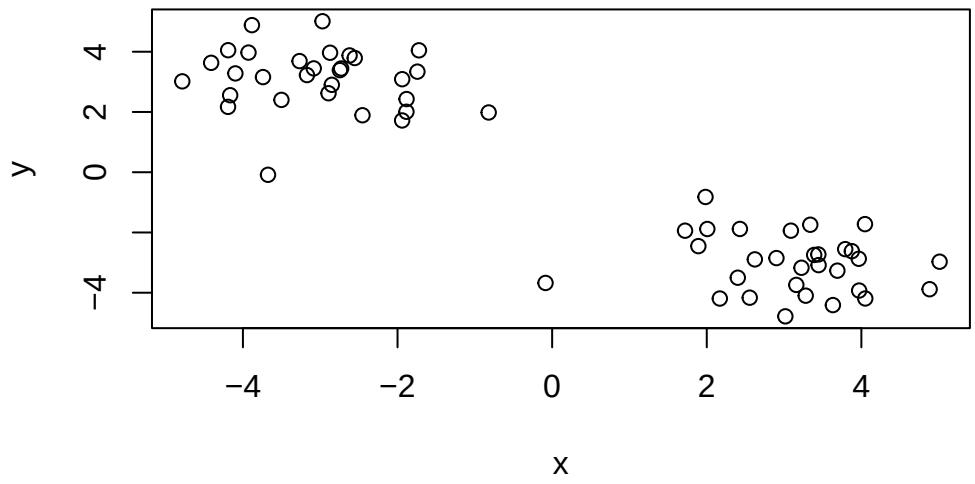
```
hist(rnorm(10000, mean=3))
```




```
x <- c(rnorm(30, -3),rnorm(30,+3))
x
```

```
 [1] -1.88487249 -2.62011001 -4.19241041 -4.78424197 -1.88295689 -1.72286721
 [7] -1.93964527 -3.67535417 -2.45352564 -0.82077599 -1.74297280 -3.08379136
[13] -4.16470312 -3.50130762 -2.87248168 -2.97053424 -4.09763135 -3.88354991
[19] -4.41078791 -2.72904317 -2.74547378 -2.55363609 -3.26738399 -3.92773368
[25] -3.74015753 -2.89275739 -4.19111878 -3.17143051 -2.85013457 -1.94027843
[31]  1.71910461  2.90130040  3.22510986  4.05030791  2.62192646  3.15822579
[37]  3.97124923  3.68949462  3.79070976  3.38932379  3.44360937  3.63155784
[43]  4.88256395  3.27988852  5.01194714  3.96602303  2.40139700  2.55591020
[49]  3.44660153  3.33894443  1.98363907  1.89138527 -0.08552161  3.08830427
[55]  4.04653868  2.42888271  3.01642194  2.16914186  3.87659147  2.00708251
```

```
z <- cbind(x=x, y=rev(x))
plot(z)
```




```
p <- 1:5
rbind(p,p)
```

```
  [,1] [,2] [,3] [,4] [,5]
p    1    2    3    4    5
p    1    2    3    4    5
```

Now we can run `kmeans()` on this input `z` and see what the results look like.

```
km <- kmeans(z, centers = 2)
km
```

```
K-means clustering with 2 clusters of sizes 30, 30

Cluster means:
          x         y
1  3.096589 -3.023789
2 -3.023789  3.096589

Clustering vector:
 [1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
Within cluster sum of squares by cluster:
[1] 57.70713 57.70713
 (between_SS / total_SS =  90.7 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
attributes(km)
```

```
$names
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"

$class
[1] "kmeans"
```

Q. How many ppoints are in each cluster

30 points in each cluster

```
km$size
```

```
[1] 30 30
```

Q What componenet of your result object details cluster assignment/membership

```
km$cluster
```

```
 [1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

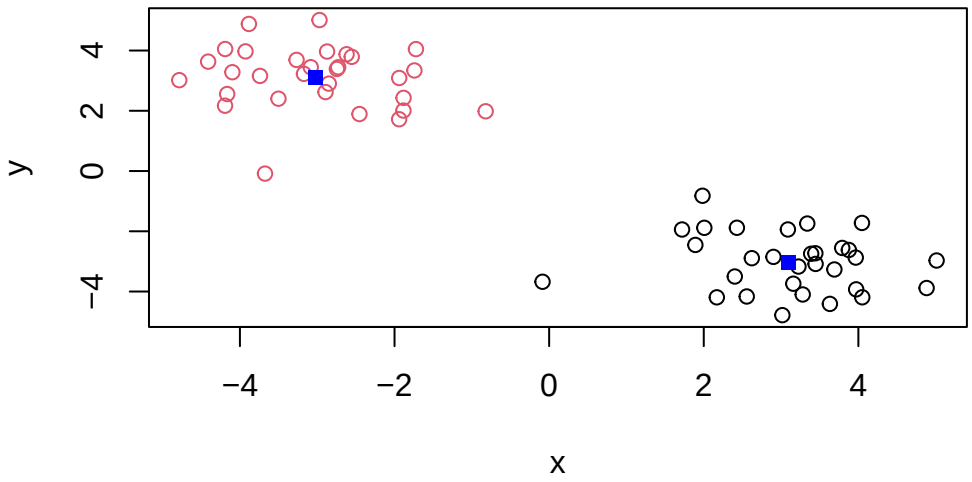
Q What componenet of your result object details cluster center

```
km$centers
```

```
          x         y
1  3.096589 -3.023789
2 -3.023789  3.096589
```

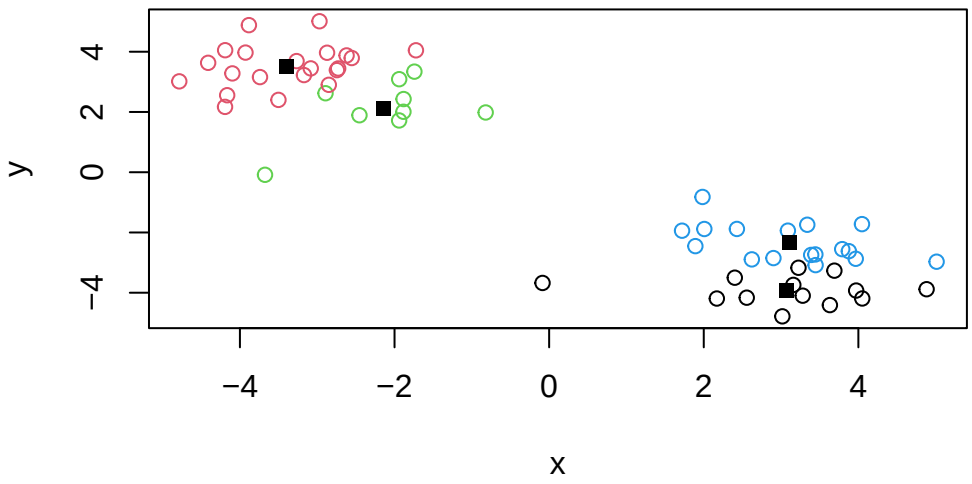
Q. Plot `z` colored by the kmeans cluster assighnment and add cluster centers as blue points

```
plot(z, col=km$cluster)
points(km$centers, col="blue", pch=15)
```



Q. Run a K-means clustering and plot the results asking for 4 clusters (k=4)?

```
km4 <- kmeans(z, centers = 4)
plot(z, col=km4$cluster)
points(km4$centers, col= "black", pch = 15)
```



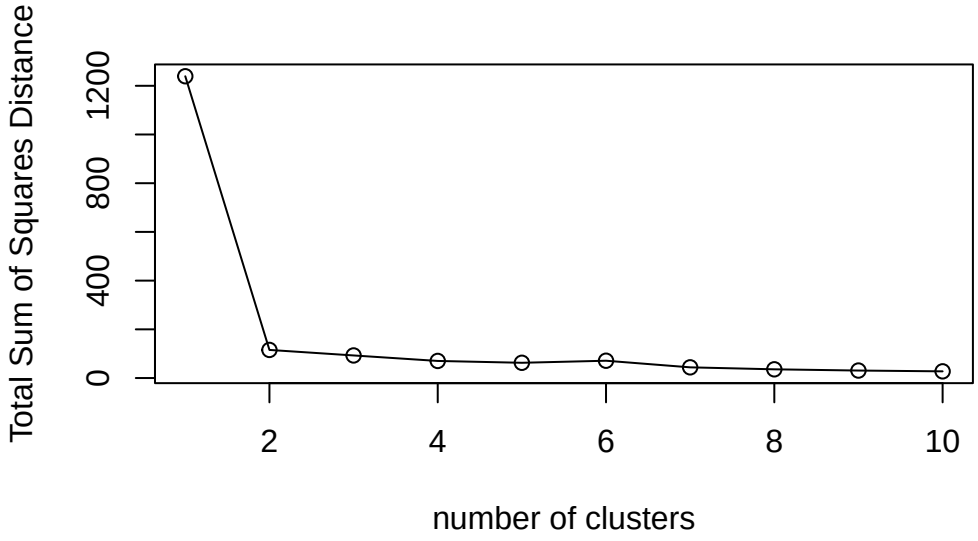
N.B You need to tell K-means the number of clusters (`CENTERS=2`)!!

One approach is to try different values for `centers` and then pick the best…

```
ans <- NULL
for(i in 1:10){
  km <- kmeans(z, centers=i)
  km$tot.withinss
  ans <- c(ans, km$tot.withinss)
}
ans
```

```
 [1] 1239.18494  115.41426   92.79979   70.18532   62.65216   71.09857
 [7]   43.92808   35.81296   30.74378   27.45646
```

```
plot(ans, typ="o",
     xlab="number of clusters",
     ylab= "Total Sum of Squares Distance")
```



Hierarchical Clustering

The main function is "base" R for hierarchical Clustering is called `hclust()`

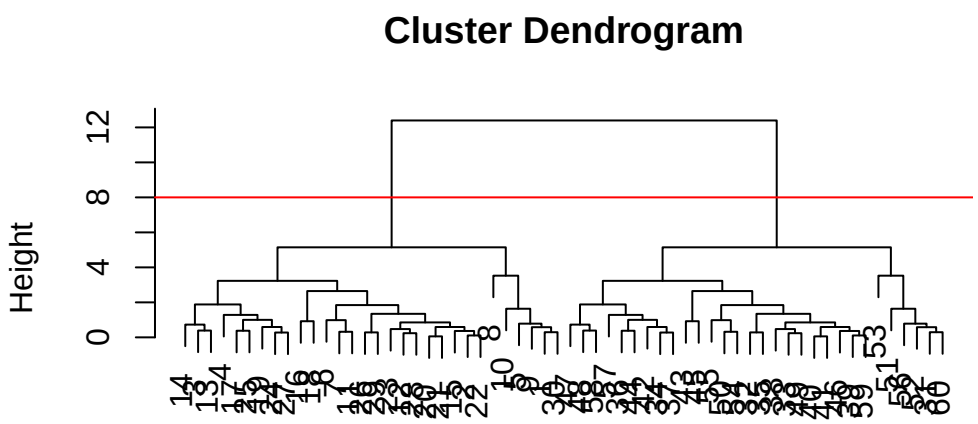
This function does not take your "raw" data for clustering, You must first build a "distance matrix" from your data and pass this as input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)
hc
```

```
Call:
hclust(d = d)

Cluster method   : complete
Distance         : euclidean
Number of objects: 60
```

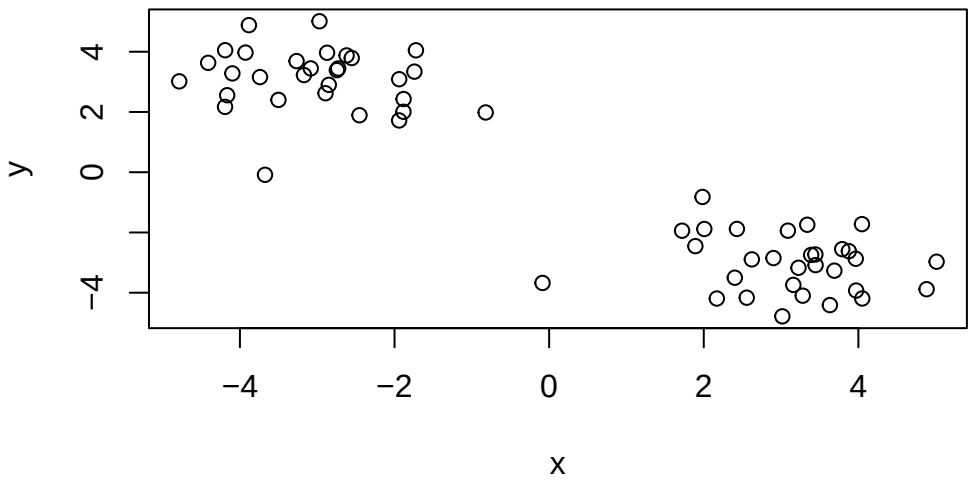
There is a bespoke `plot()` method ofr `hclust()` results object.

```r
plot(hc)
abline(h=8, col="red")
```



d
hclust (*, "complete")

```r
plot(z)
```



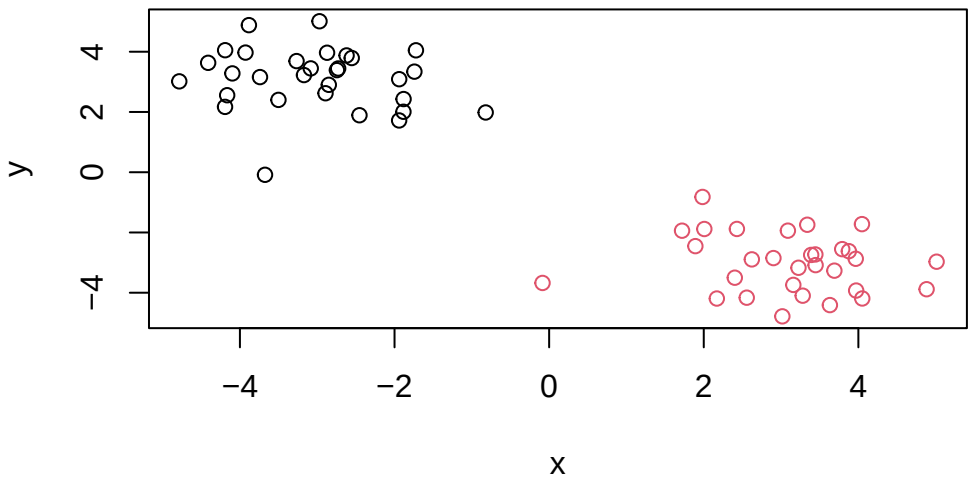
Once we have our `hclust` object (our "tree" of "cluster dendogram") we can *"cut"* the tree to reval the clustering pattern.

```
cutree(hc, h=8)
```

```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

Q. Make a plot of `z` with your hclust results(i.e colored by cluster membership)

```
grps <- cutree(hc, k=2)
plot(z, col =grps)
```



Hands on with PCA

PCA is a dimentionsallity reeducation method that is popular for revealing patterns in complex databases

Analysis of UK food

let's analyze data of the eating habits of folks from the UK to see if the patterns and trends that have some regions being distinct from others.

Data Import

Data is available in CSV format so we can use the `read.csv()` function

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
x
```

```
                X England Wales Scotland N.Ireland
1          Cheese     105   103      103        66
```

```
2         Carcass_meat      245   227      242       267
3           Other_meat      685   803      750       586
4                 Fish      147   160      122        93
5        Fats_and_oils      193   235      184       209
6                Sugars     156   175      147       139
7       Fresh_potatoes      720   874      566      1033
8            Fresh_Veg      253   265      171       143
9            Other_Veg      488   570      418       355
10 Processed_potatoes      198   203      220       187
11      Processed_Veg      360   365      337       334
12        Fresh_fruit     1102  1137      957       674
13            Cereals     1472  1582     1462      1494
14          Beverages       57    73       53        47
15        Soft_drinks     1374  1256     1572      1506
16   Alcoholic_drinks      375   475      458       135
17      Confectionery       54    64       62        41
```

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17  5
```

There are 17 rows and 5 columns

Q2. Which approach to solving the 'row-names problem' mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

```
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

```
              England Wales Scotland N.Ireland
Cheese            105   103      103        66
Carcass_meat      245   227      242       267
Other_meat        685   803      750       586
Fish              147   160      122        93
Fats_and_oils     193   235      184       209
Sugars            156   175      147       139
```

```
dim(x)
```

```
[1] 17  4
```

```
x <- read.csv(url, row.names = 1)
head(x)
```

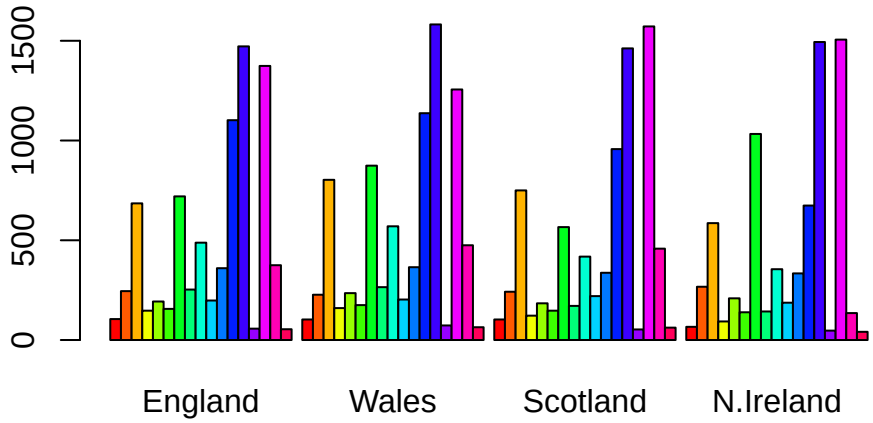
```
               England Wales Scotland N.Ireland
Cheese             105   103      103        66
Carcass_meat       245   227      242       267
Other_meat         685   803      750       586
Fish               147   160      122        93
Fats_and_oils      193   235      184       209
Sugars             156   175      147       139
```

```
dim(x)
```

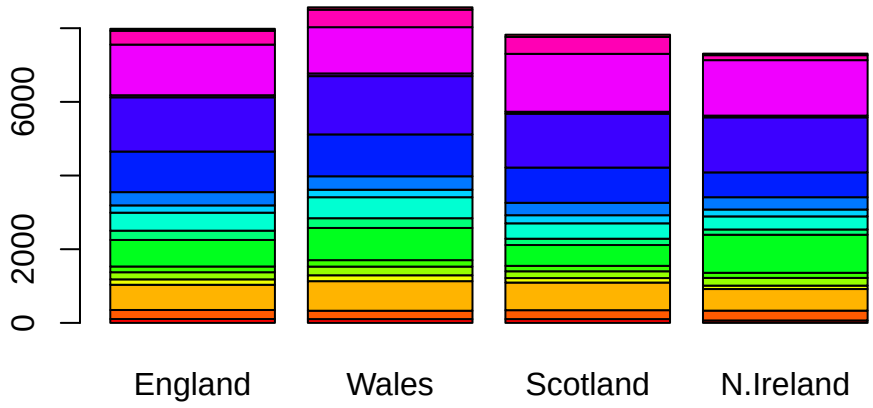
```
[1] 17  4
```

I prefer x <- read.csv(url, row.names = 1) because it imports the file right with the row names without having to manually reset it. The read.csv approach is more reliable when rerunning your code and more robust as the x <- x[,-1] also deletes columns.

```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```




```
barplot(as.matrix(x), beside= F, col=rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above barplot() function results in the following plot?

Changing beside= T to beside= F.

Q4. Changing what optional argument in the above ggplot() code results in a stacked barplot figure?

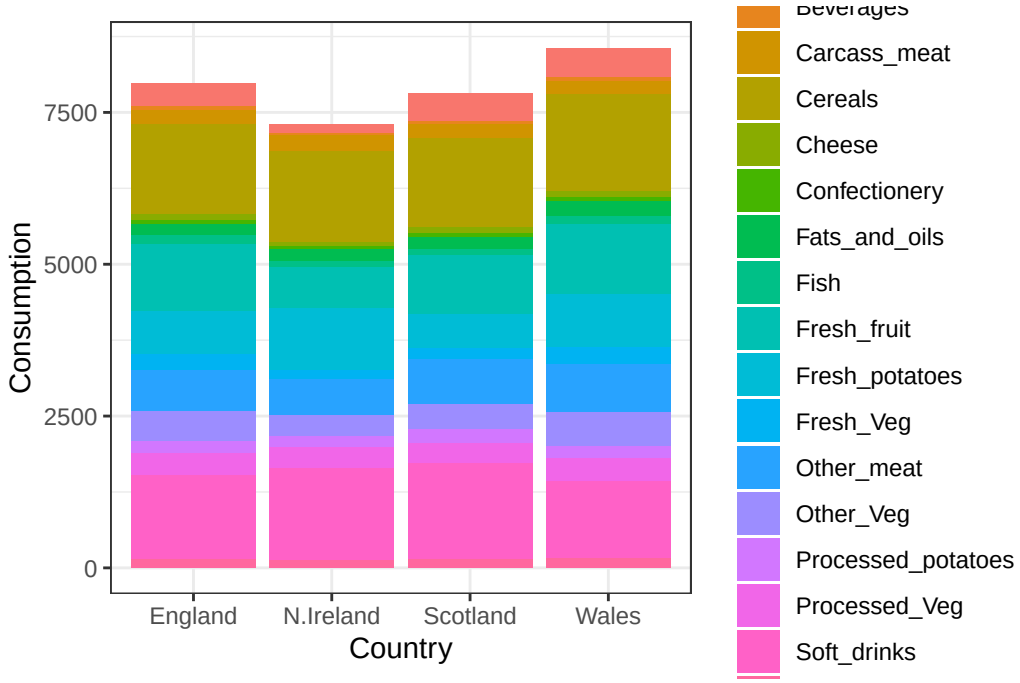
```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
          tibble::rownames_to_column("Food") |>
          pivot_longer(cols = -Food,
                       names_to = "Country",
                       values_to = "Consumption")

dim(x_long)
```

```
[1] 68  3
```

```
head(x_long)
```

```
# A tibble: 6 x 3
  Food            Country   Consumption
  <chr>           <chr>           <int>
1 "Cheese"        England           105
2 "Cheese"        Wales             103
3 "Cheese"        Scotland          103
4 "Cheese"        N.Ireland          66
5 "Carcass_meat " England           245
6 "Carcass_meat " Wales             227
```

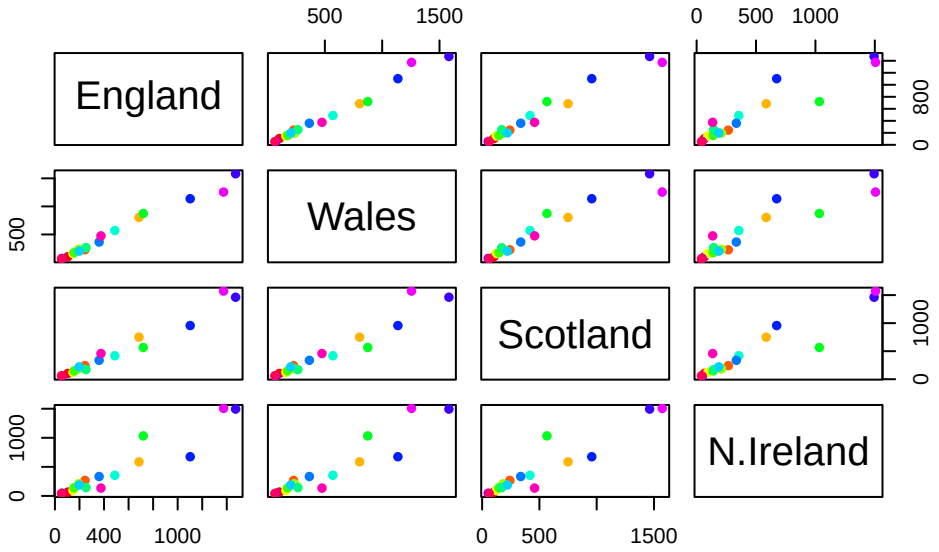
```
library(ggplot2)

ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_bw()
```



changing it from geom_col(position = "dodge") to geom_col(position = "stack") results in a stacked barplot figure.

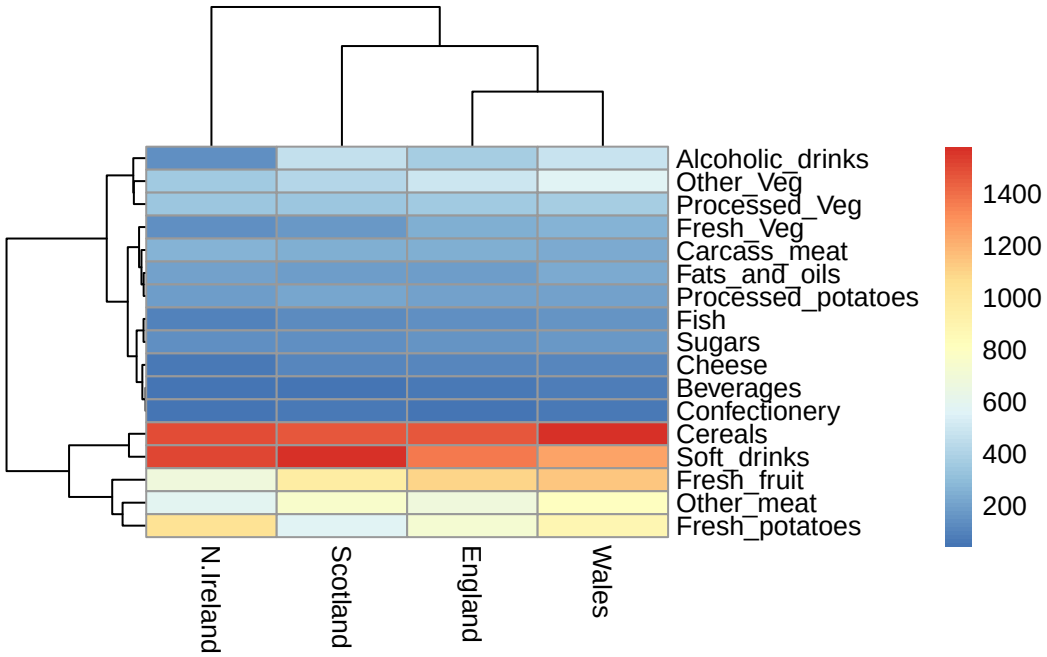
Q5. Q5: We can use the pairs() function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



The code in this figure pairs() makes scatter plots comparing each country where each point represents a certain type of food. If it lies on the diagonal it infers that the two countries being compared have the same consumption in both countries. Points that are not centered on the diagonal line means a certain country consumes it more than the other one.

```
library(pheatmap)

pheatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

England, Wales, and Scotland cluster together meaning they have similar food consumption pattern. N Ireland clusters differently as they have a higher consumption of certain items such a fresh potatoes, however using the pairs and heatmap it is unfortunately not as easy to tell and I had to thus approximate what I see as visually different.

Key-point: Even relatively small datasets can prove challenging to interpret.

PCA to the rescue

The main function in "base" R for PCA is called `prcomp()`. The function wants the "observations" to be rows and the "variables" to be columns.

```
pca <- prcomp(t(x))
summary(pca)
```

```
Importance of components:
                            PC1       PC2       PC3       PC4
```

```
Standard deviation     324.1502 212.7478 73.87622 2.921e-14
Proportion of Variance   0.6744   0.2905  0.03503 0.000e+00
Cumulative Proportion    0.6744   0.9650  1.00000 1.000e+00
```

The rturned `pca` object has components that we can use to make our main result figures.

```
attributes(pca)
```

```
$names
[1] "sdev"     "rotation" "center"   "scale"    "x"

$class
[1] "prcomp"
```

The main result figure from this analysis is called a "PC score plot" or "ordenation plot" or "PC plot" or PC1 vs PC2 plot.

The plot shows how samples relate to each other along our new acis.

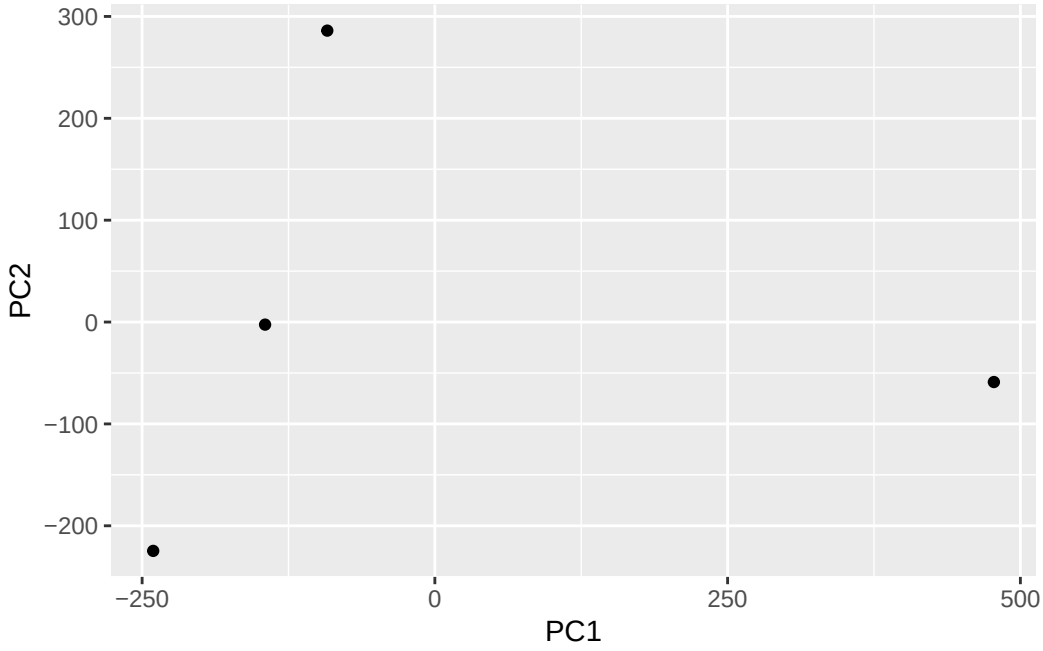
This is our new "reducded-dimensional space".

```
pca$x
```

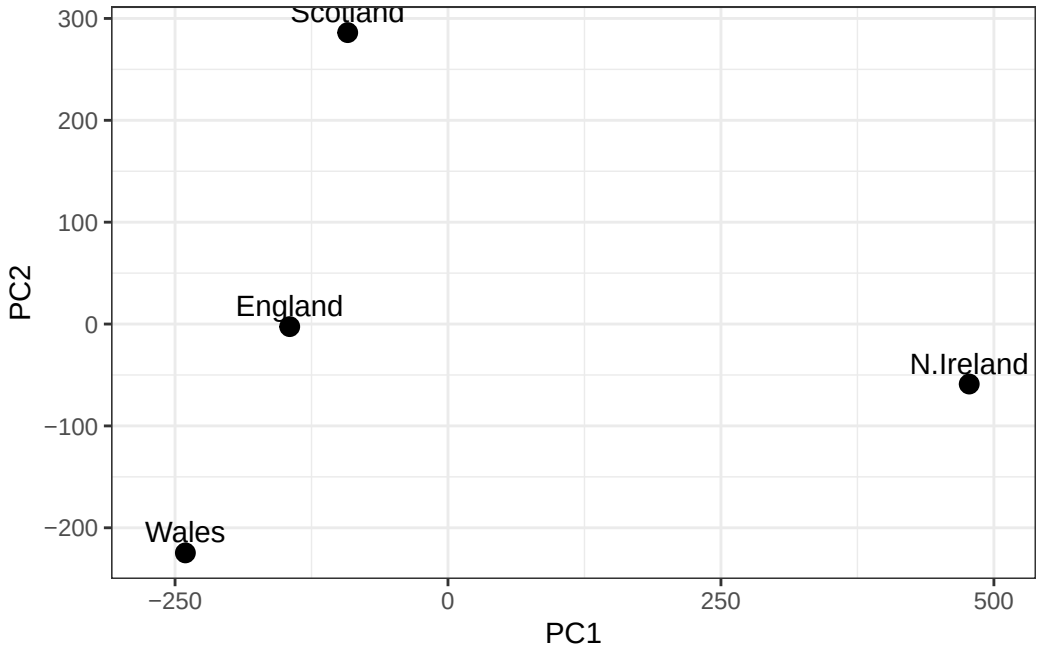
```
                PC1         PC2        PC3           PC4
England   -144.99315   -2.532999 105.768945 -9.152022e-15
Wales     -240.52915 -224.646925 -56.475555  5.560040e-13
Scotland   -91.86934  286.081786 -44.415495 -6.638419e-13
N.Ireland  477.39164  -58.901862  -4.877895  1.329771e-13
```

> Q7 . Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
library(ggplot2)

ggplot(pca$x) +
  aes(PC1, PC2) +
  geom_point()
```




```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Tidy the data

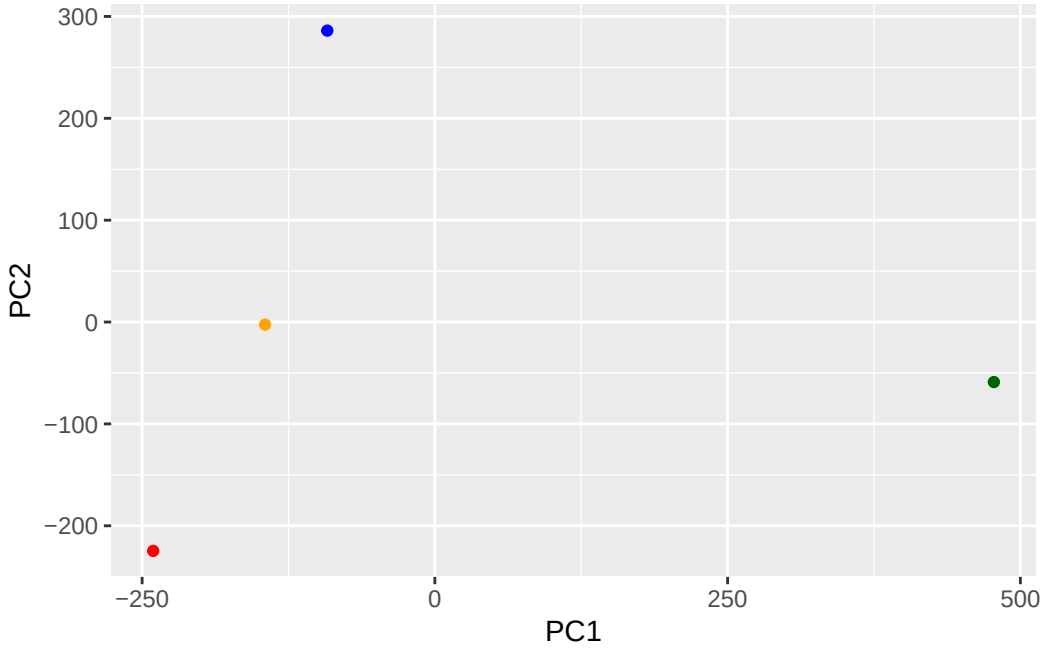
fix anything that went wrong with data import

Exporatory analysis

Make some plots to help make sense of ovious trends..

##PCA

```
mycols <- c("orange", "red", "blue", "darkgreen")

ggplot(pca$x)+
  aes(PC1, PC2) +
  geom_point(col=mycols)
```

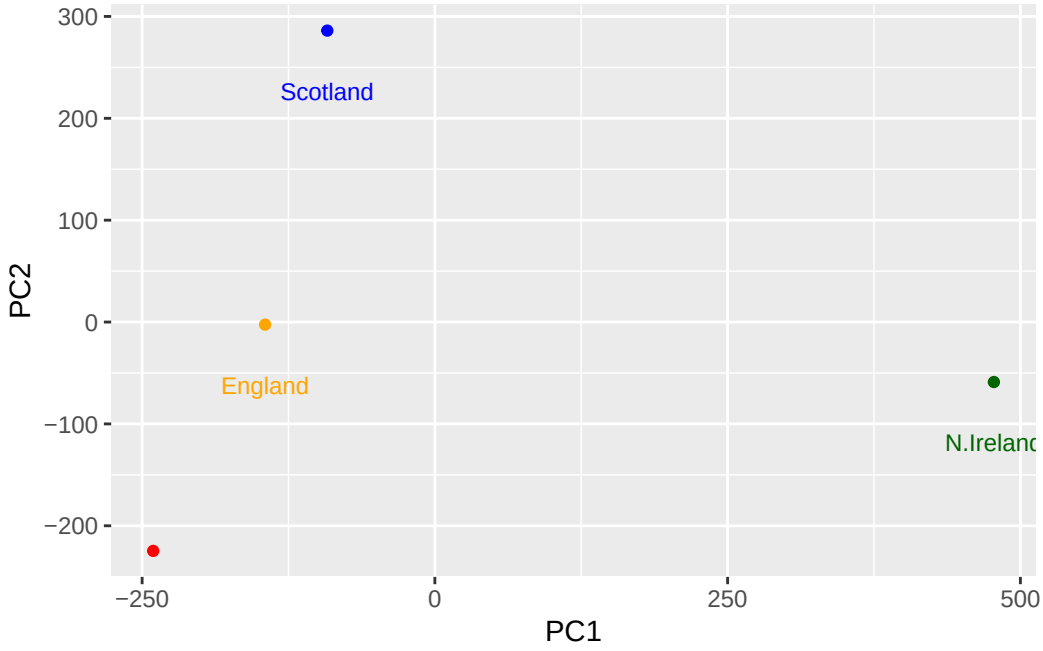


Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
pca$x
```

```
                PC1         PC2        PC3           PC4
England    -144.99315   -2.532999 105.768945 -9.152022e-15
Wales      -240.52915 -224.646925 -56.475555  5.560040e-13
Scotland    -91.86934  286.081786 -44.415495 -6.638419e-13
N.Ireland   477.39164  -58.901862  -4.877895  1.329771e-13
```

```
mycols <- c("orange", "red", "blue", "darkgreen")

ggplot(pca$x)+
  aes(PC1, PC2, label =row.names(pca$x)) +
  geom_point(col=mycols)+
geom_text(size=3, vjust=4, col=mycols)
```




```
v <- round( pca$sdev^2/sum(pca$sdev^2) * 100 )
v
```

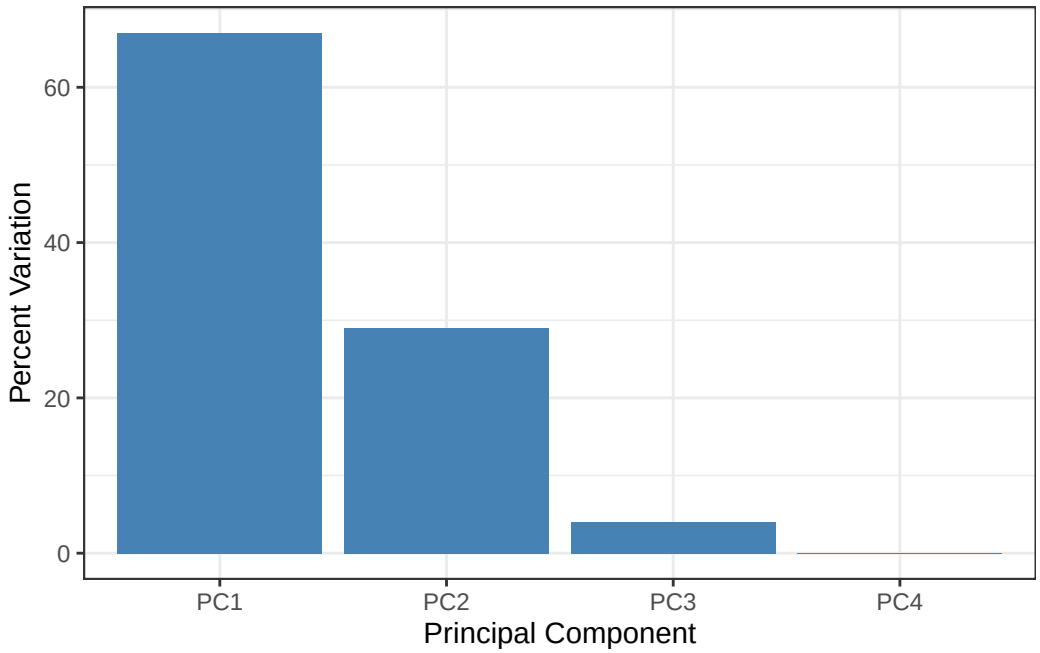
```
[1] 67 29  4  0
```

```
## or the second row here...
z <- summary(pca)
z$importance
```

```
                             PC1       PC2      PC3           PC4
Standard deviation     324.15019 212.74780 73.87622 2.921348e-14
Proportion of Variance   0.67444   0.29052  0.03503 0.000000e+00
Cumulative Proportion    0.67444   0.96497  1.00000 1.000000e+00
```

```
# Create scree plot with ggplot
variance_df <- data.frame(
  PC = factor(paste0("PC", 1:length(v)), levels = paste0("PC", 1:length(v))),
  Variance = v
)

ggplot(variance_df) +
```

```
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 0))
```

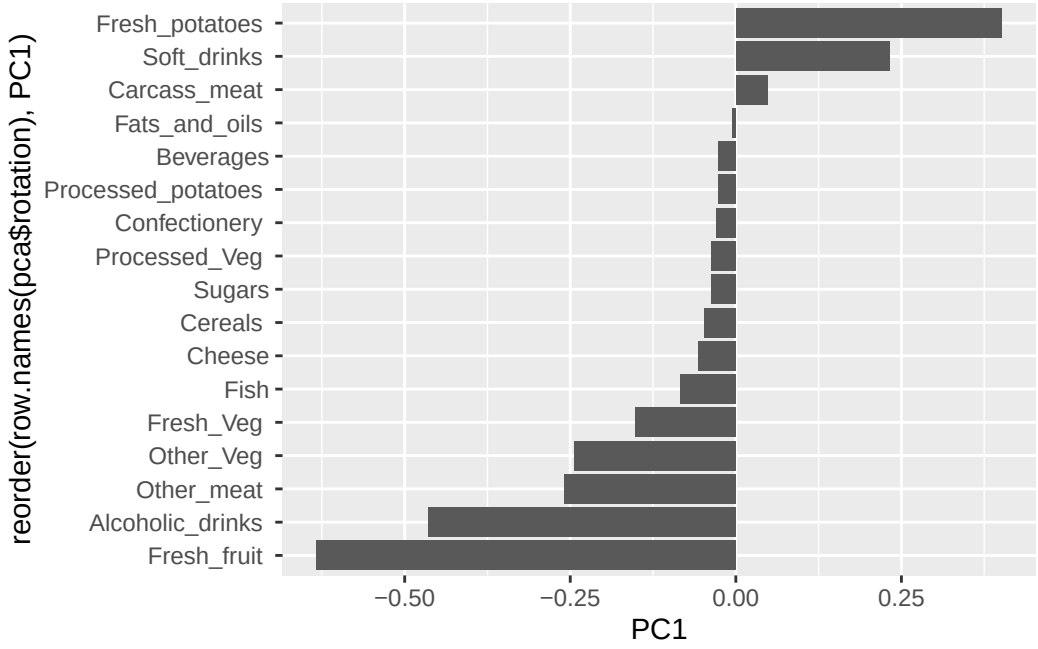



```
pca$rotation
```

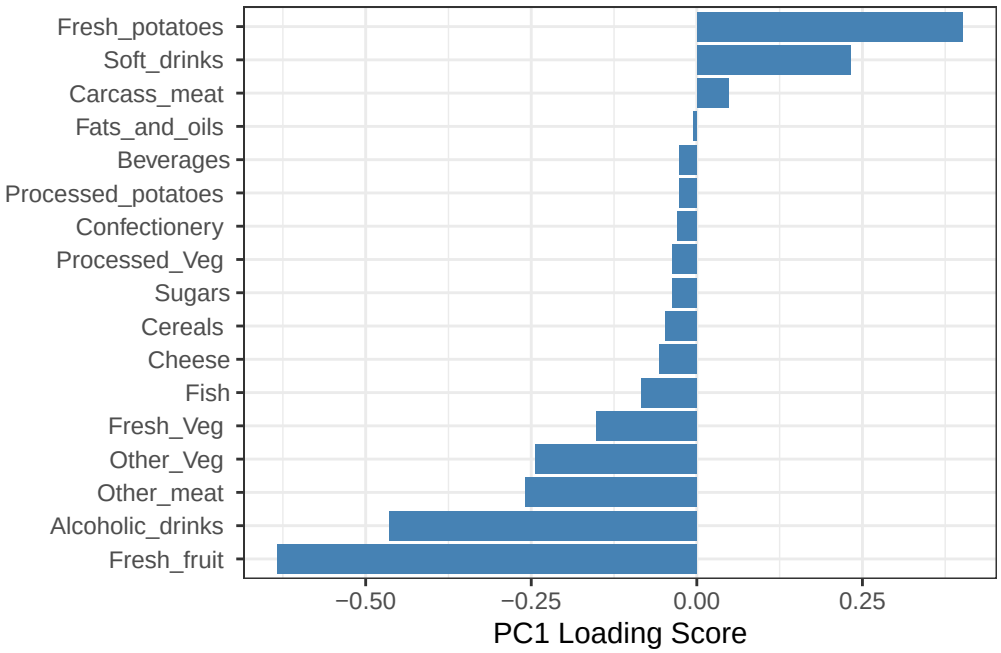
	PC1	PC2	PC3	PC4
Cheese	-0.056955380	0.016012850	0.02394295	-0.409382587
Carcass_meat	0.047927628	0.013915823	0.06367111	0.729481922
Other_meat	-0.258916658	-0.015331138	-0.55384854	0.331001134
Fish	-0.084414983	-0.050754947	0.03906481	0.022375878
Fats_and_oils	-0.005193623	-0.095388656	-0.12522257	0.034512161
Sugars	-0.037620983	-0.043021699	-0.03605745	0.024943337
Fresh_potatoes	0.401402060	-0.715017078	-0.20668248	0.021396007
Fresh_Veg	-0.151849942	-0.144900268	0.21382237	0.001606882
Other_Veg	-0.243593729	-0.225450923	-0.05332841	0.031153231
Processed_potatoes	-0.026886233	0.042850761	-0.07364902	-0.017379680
Processed_Veg	-0.036488269	-0.045451802	0.05289191	0.021250980
Fresh_fruit	-0.632640898	-0.177740743	0.40012865	0.227657348

```
Cereals             -0.047702858 -0.212599678 -0.35884921  0.100043319
Beverages           -0.026187756 -0.030560542 -0.04135860 -0.018382072
Soft_drinks          0.232244140  0.555124311 -0.16942648  0.222319484
Alcoholic_drinks    -0.463968168  0.113536523 -0.49858320 -0.273126013
Confectionery       -0.029650201  0.005949921 -0.05232164  0.001890737
```

```r
ggplot(pca$rotation) +
  aes(PC1, reorder(row.names(pca$rotation), PC1) ) +
  geom_col()
```

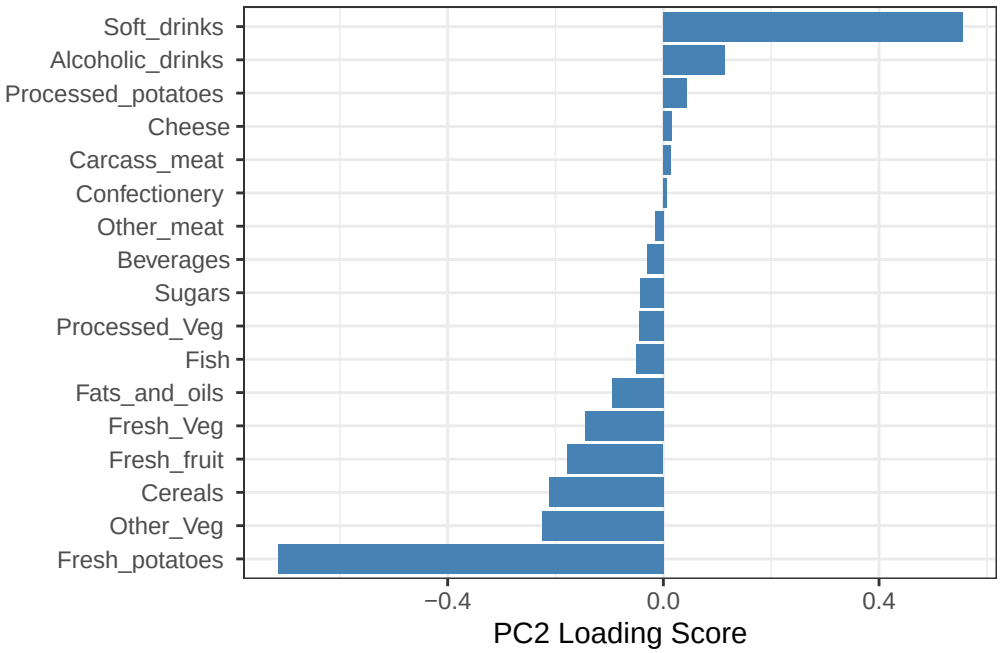



```r
## Lets focus on PC1 as it accounts for > 90% of variance
ggplot(pca$rotation) +
  aes(x = PC1,
      y = reorder(rownames(pca$rotation), PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominantely and what does PC2 maninly tell us about?

```
ggplot(pca$rotation) +
  aes(x = PC2,
      y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



The two food groups that feature prominently are soft drinks (positive) and fresh_potatoes (negative). PC2 shows the contrast between these food consumption (soft drinks and fresh poataoes)